GP2PL: From Generalized Planning Evidence to Certified BDI Plan Libraries for Temporally Extended Goals

Anonymous submission

Abstract

Generalized planning learns reusable behavior for problem families, whereas Belief–Desire–Intention (BDI) agents execute plan libraries. Direct adaptation leaves a representation gap: singleton-goal evidence may omit modules for subsidiary achievements and does not certify temporal composition. We introduce GP2PL (Generalized Planning to Plan Libraries), which compiles such evidence and Planning Domain Definition Language (PDDL) action models into certified BDI plan libraries. GP2PL validates and lifts evidence, generates schema-derived modules for producible goals, selects a compact closed atomic core, and appends query controllers guided by deterministic finite automata for finite-trace temporal goals without relearning that core. For the generated candidate language and supported fragment, accepted branches and controllers satisfy conditional soundness guarantees; uncertified obligations are rejected. Across 16 domains, five independently seeded cores achieve 98.68% mean held-out coverage (1.29 percentage-point sample standard deviation). Recursive candidates improve paired atomic success by 10.42 percentage points, and preservation certification improves paired temporal success by 9.28 percentage points. All 475 translated specifications match their gold finite-trace languages, and the fixed-core temporal study produces independently validated accepting traces for all 1,228 bound queries.

1 Introduction

Classical PDDL planning computes an action sequence for one grounded problem (McDermott et al. 1998); generalized planning learns reusable behavior for a family of related problems (Srivastava et al. 2011; Chen et al. 2026). A Belief–Desire–Intention (BDI) interpreter uses a different representation: a plan library that connects events and belief contexts to actions and subsidiary goals (Rao 1996; Meneguzzi and De Silva 2015). Turning a generalized policy into such a library is therefore a compilation problem, not a change of syntax.

In Blocks World, a learned rule for the lifted achievement `on(X,Y)` may use the action `stack(X,Y)` when the robot is holding X and the destination block is clear. If Y is not clear, however, an executable BDI library also needs a reusable plan for making it clear, even when that condition never appeared as a training goal. More generally, variables must remain safely bound, subsidiary goals must resolve to library plans, and recursive plans must make progress. Figure 1 summarizes the representation bridge and its query-reuse boundary.

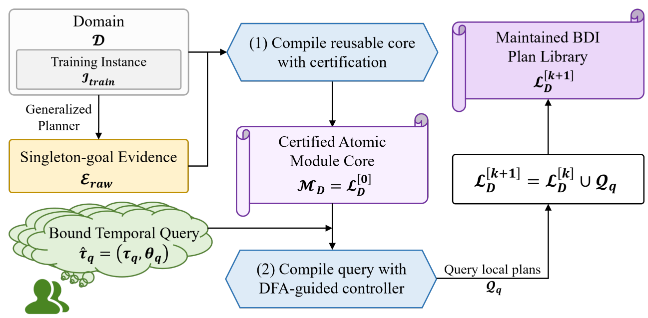

Figure 1: GP2PL separates reusable domain compilation from query-specific temporal compilation. An external generalized-planning backend derives raw singleton-goal evidence $\mathcal{E}_{\mathrm{raw}}$ from domain D and training instances $\mathcal{I}_{\mathrm{train}}$. Validated policy lifting compiles these inputs once into $\mathcal{M}_D = \mathcal{L}_D^{[0]}$; DFA-guided temporal compilation maps each bound query $\widehat{\tau}_q$ to query-local plans $\mathcal{Q}_q$, updating the sole maintained library by $\mathcal{L}_D^{[k+1]} = \mathcal{L}_D^{[k]} \cup \mathcal{Q}_q$.

Temporally extended goals add a second representation gap (De Giacomo and Vardi 2013; De Giacomo and Rubin 2018): a request to place one block on another and only later clear a third constrains the whole finite trace, and a conjunctive milestone must not be undone by a later plan—a form of BDI goal interference (Thangarajah, Padgham, and Winikoff 2003). Safe reuse therefore requires temporal control over the fixed domain library and observation after primitive actions.

The central design choice is reuse: temporal requests extend one maintained library instead of triggering a new domain-level learning run. Both compiler stages reason over typed action schemas and effects rather than domain names. Figure 2 exposes their internal transformations without mixing them with a domain-specific worked example.

GP2PL makes three contributions: it compiles singleton-goal evidence and PDDL schemas into an internally closed

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Figure 2 artwork placeholder

(a) Query-independent atomic-core compiler (b) Bound query $\rightarrow$ DFA guard $\rightarrow$ certified repair tree

Figure 2: Inside the two GP2PL compiler stages. The query-independent stage normalizes and lifts evidence, constructs and certifies schema-derived candidates, and selects $\mathcal{M}_D = \mathcal{L}_D^{[0]}$. The query-specific stage is instantiated by a bound Blocks World tower query: a MONA-derived conjunctive progress guard is converted into signed obligations, ordered by completion-effect threats, and rendered as a balanced transition-repair tree. The resulting $\mathcal{Q}_q$ is appended without relearning the core. Figure 3 separately illustrates execution of the selected atomic core.

BDI core with modules for omitted producible subgoals; it accepts only branches carrying certificates for binding, symbolic execution, achievement, completion effects, and progress; and it appends preservation-safe finite-trace controllers over the fixed core, monitoring DFA progress after each primitive action.

The experiments separate component effects from complete-system behavior: across 16 domains, independently compiled atomic cores average 98.68% held-out coverage over five evidence seeds, and a fixed-core study validates all 1,228 registered temporal queries. These claims concern the declared candidate and formula fragments, not complete planning for arbitrary PDDL–LTL$_f$ products. Formal definitions, proofs, and the reporting protocol appear in the Technical Supplement.

2 Problem Formulation and Foundations

2.1 Planning Domains and Generalized Evidence

We use typed STRIPS PDDL plus a bounded-integer resource fragment (Fikes and Nilsson 1971; McDermott et al. 1998; Fox and Long 2003). A domain is $D = \langle \mathcal{P}, \mathcal{F}, \mathcal{A}, \mathcal{T} \rangle$ with predicate schemas, integer-valued resource functions, action schemas, and a type hierarchy; a state $s = (s_{\mathcal{P}}, s_{\mathcal{F}})$ combines Boolean and integer valuations under the standard add/delete successor. The numeric component admits finite integer values, comparisons with constants, and constant integer changes; arbitrary arithmetic and continuous change are excluded. A problem instance is $I = \langle D, O_I, s_I^0, G_I \rangle$ with typed objects, initial state, and PDDL achievement condition, and a generalized-planning task supplies finite training and test instance sets sharing D. MOOSE applies goal regression (Fikes, Hart, and Nilsson 1972; Alcázar et al. 2013) to singleton goals and lifts the plans into first-order condition-to-macro rules (Chen et al. 2026). We treat these rules as evidence: they demonstrate ways to achieve observed singleton goals, but need not constitute a closed or compact atomic module core.

2.2 BDI Plan Libraries in AgentSpeak(L)

A procedural BDI plan rule has a triggering event, an applicability context, and a body (Meneguzzi and De Silva 2015). In the AgentSpeak(L) realization evaluated here (Rao 1996), an achievement plan is

$$+!p(\bar{X}) : C \leftarrow b_1; \ldots; b_m,$$

where C is a conjunction of beliefs and comparisons and each b_i is a PDDL action or an internal achievement call $!q(\bar{Y})$. A library is lifted when plans contain variables rather than training objects, and internally closed when every internal call resolves to a type-compatible selected module; closure is a signature-resolution property, not per-state applicability.

Our certificates apply to this trigger–context–body model and to the PDDL semantics of primitive actions. Formally, a candidate branch is $b = \langle g_b, C_b, \beta_b, \Sigma_b, \pi_b \rangle$: one lifted achievement literal, a conjunctive applicability condition, a finite body of primitive actions and internal calls, a conservative conditional module-completion summary, and provenance. We map selected branches to AgentSpeak(L) and evaluate them with Jason (Bordini, Hübner, and Wooldridge 2007); claims about another BDI language would require a semantics-preserving translation and an independent execution study. Query controllers follow declarative goal patterns by rechecking their intended state after subsidiary plans return (Hübner, Bordini, and Wooldridge 2006).

2.3 Finite-Trace Temporal Goals

Linear temporal logic over finite traces (LTL_f) is interpreted over a nonempty finite state sequence (De Giacomo and Vardi 2013). For proposition alphabet AP_q, the declared input language Φ_{syn} admits literals, conjunction, F, X, and strong U over AP_q, with negation restricted to atoms; the full grammar and finite-trace semantics appear in the Technical Supplement, Sec. 1. The evaluation family $\Phi_{\mathrm{bench}} \subset \Phi_{\mathrm{syn}}$ contains instances of the five predeclared profiles in Sec. 7; membership in either set is not an action-strategy guarantee.

Following automata-theoretic planning (De Giacomo and Rubin 2018), we construct the deterministic finite automaton (DFA) $\mathcal{D}_q = \langle Q_q^{\mathrm{dfa}}, 2^{AP_q}, \delta_q, q_q^0, F_q\rangle$ with LTLf2DFA and MONA (Fuggitti 2020; Henriksen et al. 1995), and a valuation map val_q interprets a PDDL state over AP_q. For automaton state z, let $d_q(z)$ be the shortest directed-path length to F_q, or ∞ when no accepting state is reachable. A progress transition (q_s, χ, q_t) satisfies $d_q(q_t) < d_q(q_s) < \infty$, and a supported guard χ has normal form

$$\chi = \bigwedge_{G \in P_\chi^+} G \wedge \bigwedge_{N \in P_\chi^-} \neg N, \qquad \mathcal{O}_\chi = \langle P_\chi^+, P_\chi^- \rangle,$$

where the signed transition obligation $\mathcal{O}_\chi$ separates positive achievements from atoms that must remain absent. MONA may represent Boolean alternatives by several valuation cubes; GP2PL treats each cube as a separate conjunctive guard, and an explicit disjunction inside one guard remains unsupported. The certificate-dependent subset $\Phi_{\mathrm{cert}}(D, \mathcal{M}_D) \subseteq \Phi_{\mathrm{syn}}$ contains the validated bound formulas whose required DFA progress obligations admit the certificates of Sec. 5; soundness claims apply to this subset, empirical claims to accepted members of Φ_{bench}. A query-local monitor evaluates the DFA initially and after every successful primitive PDDL action.

2.4 Certified Compilation Problem

Compilation problem. Given D, training instances $\mathcal{I}_{\mathrm{train}}$, and normalized singleton-goal evidence E, atomic compilation produces the certified atomic module core $\mathcal{M}_D$: the query-independent branches whose triggers are singleton achievements.

An unbound temporal specification is τ_q, its externally supplied type-consistent object binding is θ_q, and $\widehat{\tau}_q = (\tau_q, \theta_q)$ is the validated bound query. Temporal compilation produces the query-local plan set $\mathcal{Q}_q$, comprising its top-level goal, DFA dispatch, and transition-repair plans. For appended queries $q_1, \ldots, q_k$, the one maintained domain library is

$$\mathcal{L}_D^{[0]} = \mathcal{M}_D, \qquad \mathcal{L}_D^{[k]} = \mathcal{M}_D \cup \bigcup_{i=1}^{k} \mathcal{Q}_{q_i}.$$

Thus $\mathcal{M}_D$ is the stable logical core of $\mathcal{L}_D^{[k]}$, not a second persisted library, and appending $\mathcal{Q}_{q_i}$ neither reconstructs nor reoptimizes the atomic core. Throughout, calligraphic symbols denote finite sets or structured artifacts; b denotes one candidate branch, e one normalized evidence rule, and χ one DFA transition guard. The Technical Supplement, Sec. 1 gives the formal definitions, supported-fragment assumptions, and the acceptance and rejection boundary used below.

3 Related Work and Positioning

Generalized planning. Generalized planning seeks compact policies or programs that solve families of planning instances (Jiménez, Segovia-Aguas, and Jonsson 2019; Srivastava et al. 2011; Bonet and Geffner 2018; Francès, Bonet, and Geffner 2021; Bonet and Geffner 2024), via policy sketches and answer-set optimization (Drexler, Seipp, and Geffner 2022, 2024), lifted decision lists (Yang et al. 2022), or terminating programs (Segovia-Aguas, E-Martín, and Jiménez 2022); MOOSE learns lifted condition-to-macro rules by singleton-goal regression (Chen et al. 2026). Recent BDI integrations invoke a classical planner at run time (Xu et al. 2024) or use generalized planning for means–ends reasoning (Meneguzzi, Fraga Pereira, and Oren 2025). GP2PL instead addresses the representation-compilation problem: transforming singleton-goal evidence and action-schema-derived candidates into a query-independent atomic module core $\mathcal{M}_D$ that later queries extend.

Modular policies and BDI plan libraries. Policy-reuse languages let one generalized policy invoke another with parameters (Bonet, Drexler, and Geffner 2024), motivating internal calls such as `!clear(Y)`. Procedural BDI languages organize behavior as event-triggered plans selected from a context-dependent library (Meneguzzi and De Silva 2015; De Silva, Meneguzzi, and Logan 2020; Rao 1996; Bordini, Hübner, and Wooldridge 2007), with declarative-goal rechecking (Hübner, Bordini, and Wooldridge 2006), plan-failure handling (Sardina and Padgham 2007), and effect-summary interference analysis (Thangarajah, Padgham, and Winikoff 2003). GP2PL adopts these execution principles and derives certified modules, completion summaries, and declarative completion controllers from planning evidence and PDDL schemas.

Finite-trace temporal control. LTL_f admits automata-based reasoning (De Giacomo and Vardi 2013), which we instantiate with the MONA-backed LTLf2DFA construction (Fuggitti 2020; Henriksen et al. 1995). Full temporal synthesis solves the domain–automaton product (De Giacomo and Rubin 2018); GP2PL instead composes certified repairs over an existing BDI library and makes no complete product-synthesis claim. Temporally extended goals also have explicit agent-language semantics (Hindriks, van der Hoek, and van Riemsdijk 2009); we retain standard plan execution and add a query-local monitor. Following the fixed-planner principle of (Bonassi et al. 2023), FOND4LTLf (De Giacomo and Fuggitti 2021) is a task-level reference on its accepted subset, not a compiler ablation.

Natural-language temporal specifications. Prior systems map natural-language instructions to temporal logic through pattern libraries (Fuggitti and Chakraborti 2023), lifted language-to-logic translation (Chen et al. 2023; Guo, Kingston, and Kavraki 2025), or modular grounding (Liu et al. 2023); prompt-based parsers remain sensitive to input perturbations (Beurer-Kellner, Fischer, and Vechev 2023;

Zhuo et al. 2023). Translation is an evaluated input condition here, not a contribution: we validate typed, externally bound parametric LTL_f by exact finite-trace language equivalence before GP2PL compilation, with the protocol and frozen prompt in the Technical Supplement, Sec. 4.

Position of this work. Generalized planners target policies or programs, BDI programming assumes a plan library, and temporal synthesis targets an automaton strategy; GP2PL connects them through certificate-carrying compilation, echoing proof-carrying code (Necula 1997) with narrower symbolic planning obligations. Its contribution is the certified construction of a reusable atomic module core and preservation-safe query plans in the maintained BDI library $\mathcal{L}_D^{[k]}$; goal regression, answer-set solving, AgentSpeak semantics, and LTL_f-to-DFA translation remain prior components.

4 Certified Atomic-Module Compilation

4.1 Method Overview

An executable BDI library needs four properties to hold at once: every plan variable is bound, every primitive body is executable, every internal call resolves to a selected module, and every recursion makes progress. GP2PL turns each property into a compile-time, machine-checkable proof obligation and emits a structured diagnostic rather than an uncertified controller whenever an obligation fails. Building the library is therefore not transcription but construction under proof: we generate candidate branches, keep only those whose obligations discharge, and select a feasible core from what survives.

GP2PL first maps $D, \mathcal{I}_{\mathrm{train}}, \mathcal{E}_{\mathrm{raw}} \longrightarrow \mathcal{M}_D$, compiling the query-independent certified atomic module core from a PDDL domain, training split, and generalized-planning evidence. Temporal queries do not enter this optimization; every later query reuses $\mathcal{M}_D$, and the Technical Supplement, Sec. 3 gives the full pseudocode.

Certification and selection operate on trigger–context–body rules before surface rendering; AgentSpeak(L) rendering preserves the selected triggers, contexts, bodies, and internal-call relation in $\mathcal{M}_D$, and Jason provides the evaluated realization. The Technical Supplement, Sec. 1 gives the full accept/reject boundary used by the formal claims and experiments.

4.2 Evidence Normalization and Canonical Lifting

An evidence provider is admissible when its raw artifact $\mathcal{E}_{\mathrm{raw}}$ can be parsed into a provider-normalized singleton-goal evidence program E_0; canonical lifting then maps E_0 to $E = \mathrm{Lift}_D(E_0)$, whose rules pair a partial state condition and one goal atom with a PDDL action macro, numeric effects, and provenance. Admissibility requires every symbol and arity to occur in D, capture-avoiding renaming of local variables, and a singleton positive achievement target; lifting alpha-normalizes provider-local variables into typed canonical variables, preserves repeated-variable sharing across the rule, and leaves constants declared by D fixed, so a rule over `on(block0,block1)` becomes `on(X,Y)`. It does not infer a first-order policy from ground plans; that operation belongs to the external evidence provider. In the experiments, E_0 is obtained from MOOSE readable policies (Chen et al. 2026), and negative achievement evidence is rejected because those policies provide no validated action realization for a negated literal.

4.3 Action-Schema Candidate Generation

Evidence only covers goals the provider was trained on, so internal calls such as $!clear(Y)$ have no realizing rule and internal-call closure fails by construction. We therefore synthesize the missing candidates directly from the action schemas: whichever obligation must hold decides what is generated, independent of whether the provider ever observed the goal. Let $\mathrm{goalPred}(E)$ contain the predicate symbols targeted by positive predicate evidence and $\mathrm{prod}(D)$ the predicates occurring in some action's add effects; the domain-wide producible target universe is

$$T_D(E) = \mathrm{goalPred}(E) \cup \mathrm{prod}(D).$$

Thus every predicate with a positive producer receives candidate atomic modules even when the evidence provider never observed it as a goal, and no achievement target is invented for a predicate without a positive producer or validated evidence. Static predicates therefore remain context-only, and delete-only dynamic predicates are not invented as positive targets. Supported numeric equalities form $T_D^{\mathbb{Z}}(E)$, the set of admitted singleton integer-equality targets in E.

The finite candidate-construction grammar $\mathfrak{G}$ contains six constructors—producer, regression, recursion, precondition repair, resource discharge, and numeric progress—and for the Boolean and numeric targets LIFTSCHEMAS computes $\mathcal{C}_D(E) = \mathrm{Inst}_D(\mathfrak{G}, T_D(E), T_D^{\mathbb{Z}}(E))$, instantiating these constructors using only typed schemas and effects from D and alpha-normalizing every resulting variable-level branch. For target set T, the producer–precondition graph $\mathcal{G}_D^{\mathrm{prod}}(T)$ has an edge $p \to r$ when a producer for p has a positive dynamic precondition with predicate r. Backward STRIPS regression (Fikes, Hart, and Nilsson 1972; Alcázar et al. 2013; Chen et al. 2026) on an acyclic reachable graph first unifies compatible open requirements, introduces fresh variables only for unbound parameters, and rejects conflicting deletes. An alpha-normalized regression state records open requirements, producer role, and resource modes; search stops before repeating such a state and keeps the shortest body for an equivalent completion summary. Construction is therefore finite without an arbitrary action-depth constant; validated evidence macros are never truncated. Symbolic progression verifies every candidate, while cyclic dependencies require provider evidence or a separately certified recursive module. Let $\mathcal{C}_E$ be the validated evidence branches. The resulting candidate set $\mathcal{C}_{D,E} = \mathcal{C}_E \cup \mathcal{C}_D(E)$ is finite, but $\mathfrak{G}$ is not a complete hypothesis language for PDDL planning. Types constrain unification; an AgentSpeak(L) realization may expose static sort membership in contexts, but never as an achievement call or exported action.

4.4 Candidate Soundness Certificates

Every candidate carries a machine-checkable certificate. Its conditional module-completion summary Σ_b has Boolean projections $\mathrm{MustAdd}_b$, MayAdd_b, and $\mathrm{MayDelete}_b$, together with any exact constant-integer delta and keyed resource-mode transition. The $\mathrm{MustAdd}_b$ facts hold after every feasible completion; MayAdd_b and $\mathrm{MayDelete}_b$ overapproximate possible effects. Selected branches also satisfy typed binding, symbolic executability, and target achievement; $\mathrm{Cert}_D(b)$ denotes their conjunction, and the core satisfies $\mathrm{Closed}_D(\mathcal{M}_D)$. The Technical Supplement, Sec. 1 tabulates each branch constructor with its additional proof obligation and the failure it excludes; the paragraphs below give the load-bearing certificates.

Binding, symbolic execution, and internal-call closure. Action and subgoal variables must be bound by the trigger, positive context, or certified static sort membership; negative contexts and comparisons only filter. Symbolic progression proves each precondition and final trigger achievement, recording $\mathrm{MustAdd}$, MayAdd, and $\mathrm{MayDelete}$. For any BDI plan set $\mathcal{R}$, $\mathrm{Closed}_D(\mathcal{R})$ holds exactly when every internal achievement call occurring in a selected body type-unifies with the trigger of some selected plan under the type hierarchy of D. The compiler requires $\mathrm{Closed}_D(\mathcal{M}_D)$, so every internal call, including transitively reached ones, resolves to a type-compatible selected plan. This set-level property is a Clingo obligation; it proves resolution by signature and type, not applicability of some branch in every reachable state.

Well-founded recursive progress. A same-predicate recursive branch b requires a ranking function ρ_b : $\mathrm{States}(D) \to \mathbb{N}$, a non-negative relation or anchored acyclic-cone count that strictly decreases, together with an already-satisfied, direct-producer, or validated-evidence terminal branch. Threats are checked over the preparation graph selected by Clingo; a branch may add the ranked relation outside the protected cone only under a guard proving the new anchor differs from the protected one. The ranking is a compile-time witness, not an agent belief, and claims only generated schema-certified features. The construction follows relational progress witnesses in general-policy work (Francès, Bonet, and Geffner 2021).

Target-preserving resource discharge. Effects, not predicate names, identify temporary occupation of a reusable resource: when an acquisition action deletes an availability fact $\mathit{free}(R)$ and adds an occupancy relation $\mathit{held}(R,U)$ (names illustrative), the compiler infers the resource and occupant arguments from the effect structure and forms a finite graph of alpha-normalized keyed occupancy modes. A discharge body is accepted only if symbolic execution removes the current occupancy mode, restores the availability fact, leaves the atomic target true, and does not recreate an earlier normalized mode; excluding repeated modes makes the search finite without assuming that release is one action.

Ranked cross-module precondition repair. Preparing one of several dynamic preconditions may temporarily change another, such as an object's location. A repair branch may defer an unrelated sibling only through a schema-inferred single-valued fluent when every deferred sibling has one producer, all static feasibility witnesses remain, and the original target is retried before primitive production. For a candidate core $\mathcal{M}$, Clingo assigns each trigger signature in its selected call graph a dependency rank $\kappa_{\mathcal{M}}$ that strictly decreases along every selected call edge; same-predicate recursion instead requires the relational certificate above.

4.5 Feasible-Core Optimization

Validated evidence macros and certified action-schema-derived candidates enter one answer-set optimization problem solved with Clingo (Gebser et al. 2019). Evidence rules induce coverage obligations $\mathrm{covers}_D(b,e)$. Coverage requires alpha-equivalent triggers and either an equivalent body and summary, the same body under a weaker context, or a producer refinement with the same $\mathrm{MustAdd}$, no additional $\mathrm{MayDelete}$, and equivalent numeric and keyed-resource summaries. For each nonrecursive schema branch, $\mathrm{realizes}_D(b,c)$ permits an equivalent branch, a refinement with no weaker completion contract, or a target-preserving resource-discharge alternative with the same achievement contract. Body-prefix similarity alone satisfies neither relation. Define

$$\mathcal{C}^{\checkmark}_{D,E} = \{b \in \mathcal{C}_{D,E} \mid \mathrm{Cert}_D(b)\},$$
$$\mathfrak{F}_{D,E} = \{\mathcal{M} \subseteq \mathcal{C}^{\checkmark}_{D,E} \mid \mathcal{M} \models \Omega_{D,E} \wedge \mathrm{Closed}_D(\mathcal{M})\},$$

where $\Omega_{D,E}$ contains evidence coverage, schema-achievement coverage, preparation acyclicity, and recursive-ranking compatibility, and resource discharge is part of the per-branch predicate Cert_D. The optimizer uses the lexicographic objective

$$\mathcal{M}_D = \operatorname*{lexargmin}_{\mathcal{M}\in\mathfrak{F}_{D,E}} \big\langle -N_{\mathrm{rel}}(\mathcal{M}), -N_{\mathrm{prep}}(\mathcal{M}), |\mathcal{M}|, N_C(\mathcal{M}), N_\beta(\mathcal{M}) \big\rangle,$$

where N_{rel} and N_{prep} count compatible relation-ranked recursion and acyclic precondition-repair capabilities, N_C counts context literals, and N_β counts body steps. The selected trigger–context–body rules, rather than their AgentSpeak rendering, define $\mathcal{M}_D$; the claim is global only within $\mathcal{C}^{\checkmark}_{D,E}$, not over ungenerated programs.

Figure 3 isolates how the selected atomic core $\mathcal{M}_D$ is instantiated in a Blocks World state not encoded in its rules.

5 DFA-Guided Composition of Temporally Extended Goals

Validated parametric input. A temporal query is a validated parametric LTL_f specification τ_q over lifted PDDL predicates and bounded integer equalities, with a typed parameter signature and binding constraints; a runtime binding θ_q grounds it to a bound query $\widehat{\tau}_q$ whose top-level BDI goal is g_q. Predicate propositions then denote grounded Boolean facts and numeric propositions exact bounded-integer equalities in the observed state. The controlled utterances in the released benchmark define the evaluation distribution, not a

Atomic-core execution placeholder

`on(w,z),on(z,y),on(y,x)`
$\rightarrow$ `!clear(y)` $\rightarrow$ `!clear(z)`
$\rightarrow$ `!holding(y)` $\rightarrow$ `stack(y,z)`

Figure 3: Execution of the selected atomic core in Blocks World. From `on(w,z),on(z,y),on(y,x)`, the goal `!on(y,z)` calls `!clear(y)`. The same `clear/1` module recurs on `z`, decreasing the certified obstruction count before `holding/1` and the direct `stack(y,z)` producer complete the goal. The selected rules contain variables rather than these illustrative object names.

required user-facing grammar; the specification tuple, valuation map, and translation protocol appear in the Technical Supplement, Secs. 1 and 4.2. LTLf2DFA/MONA produces the DFA $\mathcal{D}_q$ of Sec. 2 and its guarded transitions (Fuggitti 2020; Henriksen et al. 1995). GP2PL attaches a controller to each progress edge (q_s, χ, q_t) with $d_q(q_t) < d_q(q_s) < \infty$; an accepting `true` self-loop needs none.

Each execution applies θ_q without grounding the shared lifted module; a binding inconsistent with the validated query specification is rejected. For a supported guard χ with signed transition obligation $\mathcal{O}_\chi = \langle P^+_\chi, P^-_\chi \rangle$ (Sec. 2), compilation computes conditional module-completion summaries and a threat-induced precedence graph, building on BDI goal-interference analyses that distinguish definite from potential plan effects (Thangarajah, Padgham, and Winikoff 2003). An acyclic graph is topologically ordered; a cyclic graph requires either an occurrence-specific preservation portfolio or a joint guard-establishment branch b^{joint}_χ whose complete net effect establishes the entire guard. The transition is rejected if neither can be certified. An accepted joint branch is callable from every positive occurrence it establishes and retains every query variable in $\mathcal{O}_\chi$. The certified serialization $\boldsymbol{\ell}_\chi$ is rendered as a balanced binary transition-repair tree (Sec. 5.3).

Figure 2(b) instantiates this construction for a bound Blocks World tower query. Its MONA progress guard requires three `on` literals to hold together; conditional completion summaries induce a bottom-up preservation-safe order, and the balanced transition-repair tree emits $\mathcal{Q}_q$ for append to $\mathcal{L}^{[k]}_D$.

5.1 Summaries and Preservation Portfolios

A relational fixed point over selected calls computes each branch summary Σ_b, and for occurrence $G_i \in P^+_\chi$ a preservation portfolio $\Pi_{\chi,i} \subseteq \mathcal{M}_D$ collects the certified branches allowed to establish G_i without disturbing the obligations protected at that position. Portfolios are certified per ordered literal occurrence rather than per predicate: two occurrences of p may need different portfolios when their protected prefixes differ. A negative guard $\neg N$ never becomes a $!\neg N$ achievement goal; it is discharged by observing absence or by an exact certified deleter. The threat-induced precedence graph $\mathcal{G}_\chi = (P^+_\chi, \prec_\chi)$ orders repairs so no established positive is later undone; the summary definition, precedence relation, and cyclic-cycle portfolio rules appear in the Technical Supplement, Secs. 1–2.

5.2 Numeric and Joint Guard Certificates

Numeric-domain support reuses the same certificate machinery. A positive integer equality is achieved by a complete constant-delta action: a unit delta requires strict direction and a larger delta the exact predecessor value. A mixed Boolean–numeric guard uses complete action-only net effects; for example, a body for $at(P, L) \wedge \mathit{fuel}(V) = 3$ must symbolically establish $at(P, L)$ and the exact integer value 3 in its final state while preserving every other signed obligation. A cyclic guard admits one joint guard-establishment action only when a single proof covers the complete successor valuation required by $\mathcal{O}_\chi$. Negated equality is observation-only without a certified change-away action; arbitrary arithmetic and uncertified sequential composition stay outside the fragment. Numeric preparation uses $\rho_{\text{num}} = \langle n_{\text{miss}}, d_{\text{target}} \rangle$, ordering missing preconditions before bounded target distance. Strong-Until source invariants are the signed literals common to every non-progress self-loop guard and are preserved until progress (Technical Supplement, Sec. 2); inserted auxiliary variables are alpha-renamed away from query and producer variables.

5.3 Balanced Binary Transition-Repair Trees

For query q, let $\mathcal{R}_{q_s,\chi}$ denote the plans for one source-state transition obligation; the query-local plan set $\mathcal{Q}_q$ contains the top-level dispatcher together with every accepted $\mathcal{R}_{q_s,\chi}$, one per progress obligation. Both a direct linear rendering and a balanced binary tree execute the same certified literal order $\boldsymbol{\ell}_\chi$; certified serialization, not the dispatch shape, provides the temporal guarantee. We adopt balanced dispatch because it bounds trigger fan-out and every generated plan body by two; the construction algorithm, completion-test pass, and suffix-safety argument appear in the Technical Supplement, Sec. 2.

The maintained library $\mathcal{L}^{[k]}_D$ is defined in Sec. 2: temporal compilation for query q_i only unions its dispatcher and transition plans $\mathcal{Q}_{q_i}$ into the single per-domain library, so query-local goals are control symbols, not PDDL fluents or exported actions. Jason records the complete primitive trace, which VAL checks against the PDDL problem (Howey, Long, and Fox 2004). If the initial valuation is accepting, the committed plan is empty and the trace is $\langle s_0 \rangle$, not an empty trace or inserted noop; a noop could change strong-Next or Until truth under finite-trace semantics (De Giacomo and Vardi 2013).

6 Conditional Guarantees

The guarantees concern accepted artifacts under their stated premises. They are conditional on the generated certified candidate set and certificate-accepted temporal subset; they do not assert complete candidate generation, universal AgentSpeak optimality, or complete strategy synthesis for arbitrary PDDL domains and DFAs.

Atomic branch soundness. **Proposition 1.** If an emitted $p(\bar{X})$ branch's context holds under a type-consistent grounding and called modules satisfy their certified achievement and completion summaries, the branch is executable and returns with $p(\bar{X})$. Binding grounds each term; induction over symbolic progression establishes preconditions and final achievement; recursion uses its ranking and preparation rechecks the producer context. □ The proposition establishes local soundness, not reachable-state coverage.

Certified-candidate-set optimality. **Proposition 2.** For the grounded selection instance defined above, a Clingo optimum $\mathcal{M}_D$ belongs to $\mathfrak{F}_{D,E}$ and minimizes the declared lexicographic cost over that feasible family. The Technical Supplement proves a bidirectional correspondence between answer sets and feasible subsets; the result does not imply complete candidate generation or optimality over all AgentSpeak programs.

Supported transition-composition soundness. **Proposition 3.** Let $\mathcal{O}_\chi = \langle P^+_\chi, P^-_\chi \rangle$ be an accepted signed transition obligation. If each required signed repair has an applicable certified branch, each selected call terminates, and later repairs preserve earlier obligations, then at the end of a complete transition-repair pass the controller retries exactly when the monitor reports the source state; otherwise it returns to dispatch at the current state. Every intervening state change is an actual DFA edge enabled by the observed valuation, and a rejecting state cannot report success.

Proof sketch. Induct over the certified order: each leaf establishes its signed literal; later summaries exclude deleting prior positives or adding forbidden atoms. If the source is left during a call, the remaining suffix still contains only these certified calls and observations, so it preserves the prefix at return boundaries. The monitor evaluates full cubes after every action and cannot be written by the controller; the suffix can therefore cause only real, fully observed DFA transitions. After the full pass, the completion plan tests the current state and either retries or returns to top-level dispatch. □
This does not imply complete action-strategy synthesis or guarantee that a post-exit suffix avoids a rejecting state.

Structural and monitor-fidelity guarantees. Three further guarantees are stated and proved in the Technical Supplement, Sec. 2. The balanced tree for n signed literals and e repair plans has $2n + e + 2$ plans, fan-out and body length at most two, and depth $O(\log n)$ while preserving the certified order (Proposition 4). After every successful primitive action, the runtime monitor state equals the deterministic automaton state over the observed finite trace, so leaving a controller source state is exactly a DFA exit and controller plans cannot write monitor beliefs (Proposition 5). If the initial valuation is accepting, successful execution commits no primitive action and denotes the one-state trace $\langle s_0 \rangle$ rather than an inserted noop, which could change strong-Next or Until truth (Proposition 6).

Complete proofs, including the binding, symbolic-progression, relational-ranking, resource-discharge, and tree-order lemmas used by these propositions, appear in the Technical Supplement, Sec. 2.

7 Experimental Evaluation

The principal comparisons are exact endpoint outcomes over heterogeneous domains rather than samples from one continuous difficulty axis. Compact main-paper tables therefore report component effects, selected-system robustness, and scope-separated external references; the Technical Supplement preserves every domain–seed cell, temporal profile, runtime distribution, and diagnostic classification.

7.1 Questions and Comparisons

Atomic compilation. Evidence Only validates provider macros against the PDDL schemas and retains them without producible-target expansion. Direct Producers adds action-only producers for the producible target universe; Maximum Feasible retains a maximum-cardinality jointly feasible subset after adding compatible recursive precondition-repair and target-preserving resource-discharge candidates; and Full GP2PL applies the lexicographic Clingo objective to the same generated certified set. Their paired comparison measures target coverage, internal-call closure, held-out execution, and compact selection of the atomic core.

Temporal composition. Unprotected Serialization uses the same automaton, library, and primitive-step monitor without completion-effect analysis. Certified Flat adds threat-induced precedence ordering and preservation portfolios; Certified Balanced changes only flat control to the balanced binary transition-repair tree. A Module-Return Monitor observes only on module return, testing the primitive-step observation boundary.

End-to-end references. End-to-end evaluation reports valid-trace coverage and keeps one-time compilation, query append, and execution costs separate. Raw MOOSE, LAMA, ENHSP MRP+HJ, and FOND4LTLf with LAMA (Chen et al. 2026; Richter and Westphal 2010; Scala et al. 2020; De Giacomo and Fuggitti 2021) are task-level references, not compiler ablations; the MOOSE comparison is partitioned by provenance before scoring, as detailed with the external references below.

7.2 Benchmarks and Protocol

Corpus and repetitions. The corpus comprises eight classical and four bounded-integer MOOSE domains with official splits (Chen et al. 2026; Chen 2026), plus four serialized-width families (Lipovetzky and Geffner 2012): Blocksworld Clear and On with published no-constants splits (Francès, Bonet, and Geffner 2021; Bonet 2026), Tower on the first quarter of a fixed IPC corpus (Bacchus 2001; Potassco 2026), and Depots on the first half of the 22-instance D2L corpus (Bonet and Geffner 2018; RLeap Project 2026). Following MOOSE (Chen et al. 2026), the atomic study uses all training instances and three goal orders to compile five independently seeded atomic cores. Source revisions, split construction, resource limits, and reporting rules appear in the Technical Supplement, Secs. 4.3 and 5.2–5.3.

Temporal benchmark construction. For each held-out problem, benchmark construction ignores the original achievement goal and derives candidate temporal milestones from legal, state-changing, nonrepeating rollouts of at most three actions; it invokes no planner. Predicate and bounded-integer changes instantiate five predeclared profiles: $F(A \wedge B)$, $F(A \wedge \neg B)$, $F(A \wedge X(F(B)))$, $F(A \wedge X(F(B \wedge X(F(C)))))$, and strong $A\,U\,B$. Each candidate retains a legal witness, and selection balances profile and semantic-signature use under a symbol-independent tie-break, yielding one witness-backed query per held-out problem: 1,228 queries over 475 distinct translation inputs. The complete construction and registered bounds appear in the Technical Supplement, Sec. 4.1.

Controlled inputs and validation. Atomic variants share each seed's evidence; temporal variants are paired on the same atomic core, binding, and DFA. A temporal success requires Jason completion (Bordini, Hübner, and Wooldridge 2007), VAL acceptance of the full primitive trace against a neutral PDDL goal (Howey, Long, and Fox 2004), and independent replay accepted by both the gold and predicted DFAs; atomic validation instead uses the original PDDL goal. Coverage is credited only to VAL-valid traces, all failures remain in the denominator, and the inference protocol with per-query outcomes appears in the Technical Supplement, Sec. 5.

7.3 Paired Component Ablations

Method	Valid (%)	Branches	KiB
Evidence Only	88.3	835.0 ± 15.8	488.3 ± 8.3
Direct Producers	88.3	1153.0 ± 15.8	549.2 ± 8.3
Maximum Feasible	**98.7**	1533.0 ± 15.8	645.9 ± 8.3
Full GP2PL	**98.7**	$\mathbf{1499.0 \pm 15.8}$	$\mathbf{635.1 \pm 8.3}$

Table 1: Paired atomic compiler comparison over 6,140 held-out seed–case executions (16 domains, 1,228 cases, five fixed seeds). All four variants compile all 80 libraries and reach the 365 producible targets except Evidence Only (90/365). Valid requires Jason success and original-goal VAL; Branches and KiB (library size) are mean $\pm$ sample standard deviation across seeds. Bold marks tied best coverage; blue bold marks Full GP2PL and its smaller core at equal coverage. Full configuration and timing appear in the Technical Supplement.

The decisive atomic change is certified recursive candidate generation: its 10.42-percentage-point gain in Table 1 concentrates entirely in the four GP2PL-added serialized-width domains. The gain survives case-level aggregation: after averaging the five seeded outcomes within each identifier, every nonzero difference favors the recursive variants, and the two-sided exact sign test gives $p = 1.47 \times 10^{-39}$, so the conclusion does not rely on treating repeated seed–case outcomes as independent. At equal coverage, Clingo's lexicographic objective yields the smallest certified library, a one-time offline compilation reused by every later query.

Method	Valid	PAR-2 s	Plans	Fan-out
Unprotected Serialization	1113/1228	342.0	7	3
Certified Flat	**1227/1228**	8.0	7	3
Certified Balanced	**1227/1228**	8.6	13	**2**
Module-Return Monitor	1211/1228	56.0	13	2

Table 2: Paired temporal compiler comparison over 1,228 queries with identical DFA, binding, and atomic-library inputs; all four build controllers for every query. Valid requires Jason, neutral-goal VAL, and acceptance by the gold and predicted DFA trace oracles. PAR-2 charges failures twice the 1,800-second limit. Plans is the median controller size and Fan-out its maximum transition-repair value. Bold marks tied best coverage; blue bold marks Balanced and its lower fan-out relative to equal-coverage Flat. Color is not used alone.

Completion-effect ordering and preservation portfolios drive the 9.28-percentage-point gain over Unprotected Serialization in Table 2, concentrated almost entirely in the four bounded-integer domains: the benefit lies in preservation-sensitive numeric composition, not uniformly across the corpus. Certified Flat and Certified Balanced share an identical success set, so balancing receives no semantic credit; we select Balanced for its structural fan-out bound, not for coverage or runtime. Moving observation from primitive steps to module return loses valid traces, chiefly through Jason timeouts, and the single shared Flat/Balanced failure is a strategy-choice boundary in which the selected branch leaves the monitor in its source state. The paired matrix conditions on its same-run seed-0 Full GP2PL library; its Certified Balanced row is an ablation estimate, and the next subsection evaluates the separately preregistered full-system configuration.

7.4 Full-System Coverage and Robustness

Five-seed atomic robustness. Across five independently compiled atomic cores, mean held-out coverage is 98.68% $\pm$ 1.29 percentage points (Table 3). The residual variation concentrates in Logistics, where seeded goal order changes the available cross-mode provider macros and the remaining producer dependency is cyclic without a decreasing certificate, and in Depots, which requires a nested support-resource choice beyond the current bound-capacity certificate. These failures delimit candidate construction; every reported success still satisfies original-goal VAL, and no domain-specific rule was introduced after observing held-out outcomes. The Technical Supplement, Sec. 5.3 gives every domain–seed diagnostic.

End-to-end temporal validation. The protocol exposes the public PDDL vocabulary, typed parameters, and binding constraints; the five registered structures instantiate Φ_{bench} rather than exhausting Φ_{syn}, and execution includes only queries admitted to $\Phi_{\text{cert}}(D, \mathcal{M}_D)$. GPT-5.5 (OpenAI 2026) translates the 475 deduplicated specifications; Table 4 reports translation and end-to-end validity. Runtime is 5.5 seconds median (interquartile range 4.28–8.49) and action count 2 median (interquartile range 1–2).

Unlike the paired estimate above, this study fixes one

Scope	Cases/seed	Valid/seed	Coverage (%)
All 16 domains	1,228	1,187–1,224	98.7 ± 1.3
All-seed complete (14)	1,127	1,127	100.0 ± 0.0
Logistics	90	54–90	86.4 ± 16.7
Depots	11	6–9	63.6 ± 11.1

Table 3: Full GP2PL atomic held-out coverage over $n = 5$ atomic cores compiled independently from predeclared evidence seeds. Coverage is mean $\pm$ sample standard deviation (SD) across seeds; Valid/seed gives the minimum–maximum count. Domains complete in every seed are aggregated, and every remaining domain is shown separately. Repeated outcomes are not pooled.

Validation unit	Valid	Total
Distinct translated specifications	475	475
Bound temporal queries	1,228	1,228

Table 4: Selected fixed-core temporal result. Translation validity requires exact finite-trace language equivalence between predicted and gold DFAs. Query validity additionally requires controller compilation, Jason completion, neutral-goal VAL, and acceptance by both DFA trace oracles. Per-profile, per-domain, runtime, and action-count distributions are reported in the Technical Supplement.

preregistered seed-0 atomic core per domain; its full coverage supports neither cross-seed claims nor arbitrary PDDL–LTL_f completeness. The translation protocol appears in the Technical Supplement, Sec. 4.2; Sec. 5.3 gives semantic-conformance cases, distributions, and provenance.

7.5 External Planning References

The published MOOSE row (five-model mean coverage from Table 4 of the extended paper (Chen et al. 2025)) is coverage-only, labelled Reported rather than locally reproduced, and its time and memory are not compared across hardware; the local Raw MOOSE extension row is labelled Measured over five models. The original and added scopes contain 1,080 and 148 cases per seed, and the source for each scope was preregistered, never selected per domain after observing outcomes. On the added serialized-width families, the descriptive same-scope contrast in Table 5 isolates the practical value of compiling evidence into a closed recursive library.

Method	Valid	Coverage (%)
Raw MOOSE evidence	117/740	15.8
Full GP2PL	**720/740**	**97.3**

Table 5: Same-scope evidence-to-library comparison on the GP2PL-added families: five predeclared seeds over the identical held-out cases. Raw MOOSE executes the provider evidence directly; Full GP2PL compiles the same seed-specific evidence with action schemas. Every valid plan passes original-goal VAL.

The published original-domain scope and the added-domain scope are disjoint, so their coverage levels are not a same-instance comparison; moreover, published Raw MOOSE coverage on its original 12-domain scope is already near ceiling. We therefore claim improved structural coverage on the added families, not universal dominance wherever Raw MOOSE already saturates. Table 5 is the only same-scope external contrast in the main paper; the Technical Supplement keeps the remaining reference scopes separate and does not rank rows drawn from different case sets.

8 Conclusion and Future Work

GP2PL closes the representation gap by converting generalized-planning evidence and PDDL action schemas into a reusable atomic BDI core, then appending query-local temporal controllers to the same maintained library. Unproved obligations fail compilation rather than becoming unchecked plans, and within the certified scope the measured component gains and 1,228/1,228 fixed-core temporal result show that certification changes both coverage and controller structure.

Several conditions delimit these results. MOOSE is the only instantiated evidence provider, the candidate-construction grammar is not a complete hypothesis language, and call closure proves type-compatible resolution rather than universal applicability. The temporal study uses controlled, witness-backed queries and a fixed seed-0 atomic core, so it does not measure unrestricted language, long-horizon planning, or cross-seed robustness, and rejected arithmetic, interference, and repair cases preclude arbitrary PDDL–LTL_f strategy synthesis.

Future work can broaden evidence acquisition through a parameterized, pinned PDDL problem generator, as in planning competitions (Bacchus 2001): a compatible generalized-planning backend can then learn evidence from validated, held-out-disjoint instance sets for the unchanged GP2PL compiler, scaling domain onboarding without extending the supported PDDL–LTL_f fragment.

References

Alcázar, V.; Borrajo, D.; Fernández, S.; and Fuentetaja, R. 2013. Revisiting Regression in Planning. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, 2254–2260. AAAI Press.

Bacchus, F. 2001. The AIPS ’00 Planning Competition. *AI Magazine*, 22(3): 47–56.

Beurer-Kellner, L.; Fischer, M.; and Vechev, M. T. 2023. Prompting Is Programming: A Query Language for Large Language Models. *Proceedings of the ACM on Programming Languages*, 7(PLDI).

Bonassi, L.; De Giacomo, G.; Favorito, M.; Fuggitti, F.; Gerevini, A. E.; and Scala, E. 2023. Planning for Temporally Extended Goals in Pure-Past Linear Temporal Logic. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 33, 61–69. Palo Alto, California: AAAI Press.

Bonet, B. 2026. Learner Policies from Examples: Benchmark Repository. GitHub repository. Revision 9991926f7655c4b6c8dc2f0404123639e42056f2.

Bonet, B.; Drexler, D.; and Geffner, H. 2024. On Policy Reuse: An Expressive Language for Representing and Executing General Policies that Call Other Policies. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, 31–39. Palo Alto, California: AAAI Press.

Bonet, B.; and Geffner, H. 2018. Features, Projections, and Representation Change for Generalized Planning. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, 4667–4673. Stockholm, Sweden: International Joint Conferences on Artificial Intelligence Organization.

Bonet, B.; and Geffner, H. 2024. General Policies, Subgoal Structure, and Planning Width. *Journal of Artificial Intelligence Research*, 80: 475–516.

Bordini, R. H.; Hübner, J. F.; and Wooldridge, M. 2007. *Programming Multi-Agent Systems in AgentSpeak Using Jason*. John Wiley & Sons.

Chen, D. Z. 2026. MOOSE Companion Benchmark Dataset. GitHub repository. Revision e00970516154e9042b783a4613a1ed7286c9beee.

Chen, D. Z.; Hofmann, T.; Klassen, T. Q.; and McIlraith, S. A. 2025. Satisficing and Optimal Generalised Planning via Goal Regression. Version 1, arXiv:2511.11095.

Chen, D. Z.; Hofmann, T.; Klassen, T. Q.; and McIlraith, S. A. 2026. Satisficing and Optimal Generalised Planning via Goal Regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 36197–36206. Palo Alto, California: AAAI Press.

Chen, Y.; Gandhi, R.; Zhang, Y.; and Fan, C. 2023. NL2TL: Transforming Natural Languages to Temporal Logics Using Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 15880–15903. Singapore: Association for Computational Linguistics.

De Giacomo, G.; and Fuggitti, F. 2021. FOND4LTLf: FOND Planning for LTLf/PLTLf Goals as a Service. In *Proceedings of the International Conference on Automated Planning and Scheduling: System Demonstrations*.

De Giacomo, G.; and Rubin, S. 2018. Automata-Theoretic Foundations of FOND Planning for LTLf and LDLf Goals. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, 4729–4735. International Joint Conferences on Artificial Intelligence Organization.

De Giacomo, G.; and Vardi, M. Y. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, 854–860. AAAI Press.

De Silva, L.; Meneguzzi, F.; and Logan, B. 2020. BDI Agent Architectures: A Survey. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, 4914–4921. International Joint Conferences on Artificial Intelligence Organization.

Drexler, D.; Seipp, J.; and Geffner, H. 2022. Learning Sketches for Decomposing Planning Problems into Subproblems of Bounded Width. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 32, 62–70. Palo Alto, California: AAAI Press.

Drexler, D.; Seipp, J.; and Geffner, H. 2024. Expressing and Exploiting Subgoal Structure in Classical Planning Using Sketches. *Journal of Artificial Intelligence Research*, 80: 171–208.

Fikes, R. E.; Hart, P. E.; and Nilsson, N. J. 1972. Learning and Executing Generalized Robot Plans. *Artificial Intelligence*, 3: 251–288.

Fikes, R. E.; and Nilsson, N. J. 1971. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artificial Intelligence*, 2(3–4): 189–208.

Fox, M.; and Long, D. 2003. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *Journal of Artificial Intelligence Research*, 20: 61–124.

Francès, G.; Bonet, B.; and Geffner, H. 2021. Learning General Planning Policies from Small Examples Without Supervision. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 11801–11808. Palo Alto, California: AAAI Press.

Fuggitti, F. 2020. LTLf2DFA: Translate LTLf and PLTLf Formulae to Deterministic Finite Automata.

Fuggitti, F.; and Chakraborti, T. 2023. NL2LTL—A Python Package for Converting Natural Language Instructions to Linear Temporal Logic Formulas. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 16428–16430. AAAI Press.

Gebser, M.; Kaminski, R.; Kaufmann, B.; and Schaub, T. 2019. Multi-shot ASP Solving with Clingo. *Theory and Practice of Logic Programming*, 19(1): 27–82.

Guo, W.; Kingston, Z.; and Kavraki, L. E. 2025. CaStL: Constraints as Specifications Through LLM Translation for Long-Horizon Task and Motion Planning. In *2025 IEEE International Conference on Robotics and Automation*, 11957–11964. IEEE.

Henriksen, J. G.; Jensen, O. J. L.; Jørgensen, M. E.; Klarlund, N.; Paige, R.; Rauhe, T.; and Sandholm, A. B. 1995. MONA: Monadic Second-Order Logic in Practice. *BRICS Report Series*, 2(21).

Hindriks, K. V.; van der Hoek, W.; and van Riemsdijk, M. B. 2009. Agent Programming with Temporally Extended Goals. In *Proceedings of the 8th International Conference on Autonomous Agents and Multiagent Systems*, 137–144. International Foundation for Autonomous Agents and Multiagent Systems.

Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence*, 294–301. IEEE Computer Society.

Hübner, J. F.; Bordini, R. H.; and Wooldridge, M. 2006. Programming Declarative Goals Using Plan Patterns. In *Declarative Agent Languages and Technologies IV*, volume 4327 of *Lecture Notes in Computer Science*, 123–140. Springer.

Jiménez, S.; Segovia-Aguas, J.; and Jonsson, A. 2019. A Review of Generalized Planning. *The Knowledge Engineering Review*, 34: e5.

Lipovetzky, N.; and Geffner, H. 2012. Width and Serialization of Classical Planning Problems. In *Proceedings of the 20th European Conference on Artificial Intelligence*, volume 242 of *Frontiers in Artificial Intelligence and Applications*, 540–545. IOS Press.

Liu, J. X.; Yang, Z.; Idrees, I.; Liang, S.; Schornstein, B.; Tellex, S.; and Shah, A. 2023. Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, 1084–1110. PMLR.

McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL—The Planning Domain Definition Language.

Meneguzzi, F.; and De Silva, L. 2015. Planning in BDI Agents: A Survey of the Integration of Planning Algorithms and Agent Reasoning. *The Knowledge Engineering Review*, 30(1): 1–44.

Meneguzzi, F.; Fraga Pereira, R.; and Oren, N. 2025. Generalised BDI Planning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, 1483–1491. International Foundation for Autonomous Agents and Multiagent Systems.

Necula, G. C. 1997. Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–119. ACM.

OpenAI. 2026. GPT-5.5 System Card. OpenAI system card.

Potassco. 2026. PDDL Instances from the International Planning Competitions. GitHub repository. Revision cf19edf7c53d1540ddbb396c642595e0926ee552.

Rao, A. S. 1996. AgentSpeak(L): BDI Agents Speak Out in a Logical Computable Language. In *Agents Breaking Away*, Lecture Notes in Computer Science, 42–55. Berlin, Heidelberg: Springer Berlin Heidelberg.

Richter, S.; and Westphal, M. 2010. The LAMA Planner: Guiding Cost-Based Anytime Planning with Landmarks. *Journal of Artificial Intelligence Research*, 39: 127–177.

RLeap Project. 2026. D2L: Description Logic Features for Planning. GitHub repository. Revision 0620e169c894d79b3c84f435dba1462996f7c270.

Sardina, S.; and Padgham, L. 2007. Goals in the Context of BDI Plan Failure and Planning. In *Proceedings of the 6th International Joint Conference on Autonomous Agents and Multiagent Systems*, 1–8. ACM.

Scala, E.; Saetti, A.; Serina, I.; and Gerevini, A. E. 2020. Search-Guidance Mechanisms for Numeric Planning Through Subgoaling Relaxation. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30, 226–234. AAAI Press.

Segovia-Aguas, J.; E-Martín, Y.; and Jiménez, S. 2022. Representation and Synthesis of C++ Programs for Generalized Planning. *CoRR*, abs/2206.14480.

Srivastava, S.; Immerman, N.; Zilberstein, S.; and Zhang, T. 2011. Directed Search for Generalized Plans Using Classical Planners. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 21, 226–233. AAAI Press.

Thangarajah, J.; Padgham, L.; and Winikoff, M. 2003. Detecting and Avoiding Interference Between Goals in Intelligent Agents. In *Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence*, 721–726. Morgan Kaufmann.

Xu, M.; Lumley, T.; Fraga Pereira, R.; and Meneguzzi, F. 2024. A Practical Operational Semantics for Classical Planning in BDI Agents. In *ECAI 2024—27th European Conference on Artificial Intelligence*, volume 392 of *Frontiers in Artificial Intelligence and Applications*, 1365–1372. IOS Press.

Yang, R.; Silver, T.; Curtis, A.; Lozano-Pérez, T.; and Kaelbling, L. P. 2022. PG3: Policy-Guided Planning for Generalized Policy Generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, 4686–4692. Vienna, Austria: International Joint Conferences on Artificial Intelligence Organization.

Zhuo, T. Y.; Li, Z.; Huang, Y.; Shiri, F.; Wang, W.; Haffari, G.; and Li, Y.-F. 2023. On Robustness of Prompt-Based Semantic Parsing with Large Pre-Trained Language Model: An Empirical Study on Codex. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 1090–1102. Dubrovnik, Croatia: Association for Computational Linguistics.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this `.tex` file directly.

For each question (that applies), replace the "Type your response here" text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the "Type your response here" text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this .tex file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference's website to see if you will be asked to provide this checklist with your paper or separately.

---

1. General Paper Structure

1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) yes

1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) yes

1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) yes

2. Theoretical Contributions

2.1. Does this paper make theoretical contributions? (yes/no) yes

If yes, please address the following points:

2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) yes

2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) yes

2.4. Proofs of all novel claims are included (yes/partial/no) yes

2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) yes

2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) yes

2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) yes

2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) yes

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) yes

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) yes

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) yes

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) yes

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) yes

3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) yes

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisficing (yes/partial/no/NA) yes

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) yes

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) yes

4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) yes

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) yes

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) yes

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) yes

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) yes

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) yes

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) yes

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) yes

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) yes

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) yes

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) yes